

Software Implementation and Testing Document

For

Group 14

Version 2.0

Authors:

Samantha Bui
Ludginie Dorval
Antonio Garriga
William Lee
Lillian Malik

1. Programming Languages (5 points)

1. **Python:** Employed in both back-end and front-end development, as well as for database integration. Python was chosen due to its versatility, extensive library support, and ease of use. Specifically, Python Flask and Encryption libraries.
2. **SQL:** Utilized for managing and interacting with the back-end database. SQL was selected for its reliability and its ability to handle database management.
3. **HTML:** Used for structuring the front-end user interface. HTML was chosen because it is the standard language for web page design. In increment 3, we will also be using CSS to enhance the visual aspect of the project.
4. **JSON:** Used for data formatting and communication between the back-end and front-end components. JSON was selected for its simplicity and efficiency in representing complex data structures, as well as its compatibility across different languages.

2. Platforms, APIs, Databases, and other technologies used (5 points)

1. **MySQL:** Implemented as the back-end database management system.
2. **Flask:** Utilized for the development of the front-end framework and for facilitating secure database interactions. Flask was selected for its versatility in integration.
3. **Cipher:** Integrated for front-end encryption and secure data handling. Cipher was chosen due to its strong encryption capabilities.
4. **JSON:** Used for data formatting and communication between the back-end and front-end components.
5. **Virtual Studio Code:** We used this Integrated Development Environment (IDE) for writing, debugging, and running code.
6. **GitHub:** Used for team collaboration, providing easy access to source code and documentation.

3. Execution-based Functional Testing (10 points)

1. **Cost-Volume-Profit (CVP) Analysis:** By testing and verifying calculations.
2. **Production Budgeting (BP):** Tested budgeting formulas to ensure correct inventory predictions.
3. **Role-Based Access Control (RBAC):** Ensured proper user access based on assigned roles and restricted unauthorized actions. Tested by running Flask and logging in with initialized admin, user, and viewer. The front-end logic was tested independently from the back-end logic.
4. **Encryption:** Confirmed to function properly. Passwords and sensitive user data are securely encrypted before storage. Username admin and password admin logs the tester in. Username user and password user logs the tester in. Username viewer and password viewer logs the tester in. Very good progress and compiles, showing user interface. Just need to implement the different functions.
5. **User Interface Integration:** After login, the application correctly renders role-specific dashboard buttons for Admin, User, and Viewer roles. All navigation links work as expected and route to the proper templates.

4. Execution-based Non-Functional Testing (10 points)

We performed basic execution-based testing to confirm several non-functional requirements listed in our RD document. We verified that the application loads quickly and runs smoothly during typical use, including logging in and navigating between pages. We tested role-based access by logging in with different accounts (admin, user, viewer) to make sure each user only sees what they are allowed to. We also confirmed that passwords are encrypted and decrypted correctly during login. To test portability, we cloned the project on another machine and confirmed that the app worked without needing manual database setup.

Finally, we checked for reliability by attempting to access restricted routes without logging in, and the app correctly redirected to the login page without crashing.

5. Non-Execution-based Testing (10 points)

In this increment, due to the limited three-week timeframe, our focus was primarily on testing the code and finalizing the HTML files. Unlike Increment 1, which spanned eight weeks, we had to streamline our efforts to ensure the core functionalities were working properly within the shorter period. We conducted only one meeting this time, due to the time constraints, whereas in Increment 1, we held multiple bi-monthly meetings. Despite the reduced number of meetings, we still reviewed progress and set clear objectives. Meeting notes and updates regarding our coding and repository modifications can be found in the GitHub README file, as was done previously.

We have communicated through issues to ensure accuracy and consistency in integration of the front-end code as well as the back-end code synchronously through in person meetings and asynchronously to ensure we are making progress. We have committed code and closed issued on GitHub.

We also focused on detailed non-execution-based debugging and validation during the final stages of the increment. We paid close attention to verifying that all Flask routes correctly mapped to their respective HTML templates, and that session variables (such as `session['security']`) were used consistently between the Python backend and the Jinja logic in the HTML files.

We identified and resolved errors between route names and template references by correcting issues such as `url_for('view_data')` to `url_for('list_data')`. We also ensured that the home dashboard responded dynamically to different user roles (Admin, User, Viewer). These tests were completed through code tracing, logic checks, and visual walkthroughs of the rendered pages—without the need for execution-based testing.

To double-check integration, we pulled the latest version from GitHub after changes were merged and re-ran the application locally, confirming that the functionality matched what was intended in the codebase. This served as a final validation that our non-execution-based checks were accurate and that our collaborative workflow was effective.